How It Works Underneath

The data path, the query logic, the architectural decisions and the reasoning behind each one — written so that any engineer can review, extend or challenge the system.

Companion to the project record **Covers** serve.py 935 lines · dashboard_updated.html 1,589 lines
Date 15 September 2026

00 · Orientation

What this document is

The project record explains *what* was delivered and why the numbers can be trusted. This document explains *how*: the schema the data passes through, the SQL each screen runs, the decisions taken in the server, and the reasoning that produced them.

Every claim here is traceable to code or to the database catalogue. Where a design choice has a cost as well as a benefit, both are stated.

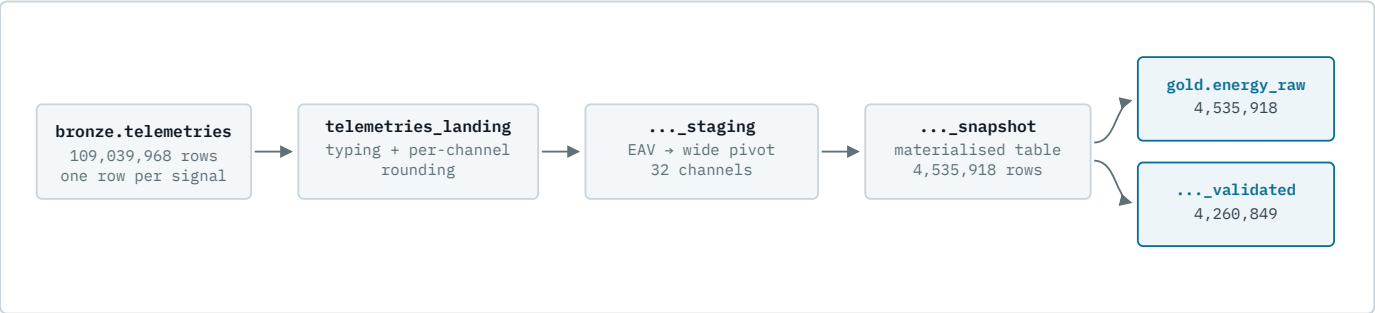
THE ONE IDEA THE WHOLE SYSTEM RESTS ON

The dashboard **never calculates consumption**. It reads a column the warehouse already computed, because the business rules for what counts as valid consumption live in the warehouse, not in the presentation layer. Every other decision below follows from that.

01 · Lineage

The path a number takes

Five hops from a meter register to a pixel. Each one is a real object in the database that can be queried independently.



Hop 1 — bronze.telemetries is not tabular

The source is an EAV (entity–attribute–value) table. One row is *one measurement of one channel at one instant*, not one meter reading:

```
device_id  uuid     -- equals energy_meter_key; no mapping table needed
key_id     bigint   -- FK to bronze.telemetries_dictionaries (Amp1, V1, PF2, TkW...)
timestamp  bigint   -- epoch MILLISECONDS, not a timestamp type
dbl        double   -- the value; `long` is used for integer channels
PRIMARY KEY (device_id, key_id, timestamp)
```

One meter reading is therefore ~24 rows. 4.5M readings × 24 channels ≈ the 109M row count. The primary key ordering `(device_id, key_id, timestamp)` matters later: any verification query must lead with `device_id` to use the index.

Hop 2 — landing applies per-channel rounding

`silver.telemetries_landing` is a view, and it is **not** a passthrough. It quantises each channel to the precision that channel is meaningful at:

```
CASE
  WHEN td.key IN ('V1','V2','V3','Vn1','Vn2','Vn3','Va','Vna')
                                    THEN round(dbl::numeric, 1)
  WHEN td.key IN ('Amp1','Amp2','Amp3','Ampa')  THEN round(dbl::numeric, 1)
  WHEN td.key = 'Hz'                            THEN round(dbl::numeric, 2)
  WHEN td.key IN ('PF1','PF2','PF3','PFa')      THEN round(dbl::numeric, 2)
  WHEN td.key IN ('kW1','kW2','kW3','TkW')      THEN round(dbl::numeric, 3)
  WHEN td.key IN ('kVAr1','kVAr2','kVAr3','TkVAr')  THEN round(dbl::numeric, 1)
  WHEN td.key IN ('kWh','kVArh','kVAh')             THEN round(dbl::numeric, 1)
  WHEN td.key IN ('kVA1','kVA2','kVA3','TkVA')      THEN round(dbl::numeric, 3)
  ELSE round(dbl::numeric, 9)
END
```

Verified during validation: for the meters tested this rounding is a no-op — the devices already transmit at these precisions — so it constrains future sources rather than altering current data.

Hop 3 — staging pivots EAV into a wide row

The pivot is a conditional aggregate. Each of 32 channels becomes a column via `max() FILTER`, grouped by meter and instant:

```
max(numeric_reading_value) FILTER (WHERE telemetry_key = 'Amp1') AS current_phase_1_a,
max(numeric_reading_value) FILTER (WHERE telemetry_key = 'TkW')  AS active_power_total_kw,
...
GROUP BY energy_meter_key, source_timestamp_ms, ...
```

WHY MAX() AND NOT, SAY, AVG()

Within one `(meter, timestamp)` group each channel should appear exactly once, so the aggregate is only there to collapse the group — `max()` is the cheapest way to say "take the single value". The view also emits `duplicate_measurement_row_count` so a genuine duplicate is *counted* rather than silently averaged away. Averaging would have hidden the fault.

This is also where the Saudi time columns are derived, using a named zone rather than an offset: `to_timestamp(ms/1000.0) AT TIME ZONE 'Asia/Riyadh'`.

Hop 4–5 — snapshot, then the gold views

`silver.fact_energy_readings_snapshot` is a materialised **table**, not a view — the pivot is far too expensive to run per query. `gold.energy_raw` is then a thin view over it (same 4,535,918 rows), and `gold.energy_consumption_validated` is the consumption model described next.

---

02 · Consumption

The model the dashboard reads

This is the layer that encodes business judgement, and the reason the dashboard does no arithmetic of its own.

A cumulative register cannot simply be differenced. Meters reset, drop out, report implausible jumps, and are commissioned mid-window. The gold layer resolves each of those into two enumerated columns that travel with every interval:

COLUMN	VALUES	MEANING
`interval_quality`	NORMAL_INTERVAL	Consecutive readings at the expected cadence
	LONG_GAP	Energy accrued across a longer-than-normal interval — still real consumption
	FIRST_VALID_READING	No predecessor to difference against
	IGNORED_READING	Excluded by rule
	UNRESOLVED_DECREMENT	Register went backwards and could not be explained
`consumption_cleaning_status`	KEEP	Counts toward the total
	ISOLATED_LOW_BAD ISOLATED_HIGH_BAD	Single-sample spikes in either direction
	STARTUP_EXCLUDED	Commissioning period

The resulting column, `validated_kwh_consumption`, is what both the dashboard and Power BI sum. Nothing downstream re-implements any of this:

```
SELECT sum(validated_kwh_consumption) FROM gold.energy_consumption_validated;
  2020697.5
   = NORMAL_INTERVAL 1,666,455.0  +  LONG_GAP 354,242.5
```

TWO FACTS THAT SURPRISE PEOPLE, AND ARE CORRECT

LONG_GAP energy is included (17.5% of the total). A gap in *reporting* is not a gap in *consumption* — the register kept counting. Excluding it would understate the bill. This is a deliberate warehouse decision, not an oversight.

The validated view has 4,260,849 rows and 29 meters, while raw has 4,535,918 and 31. Zone 13's two meters are excluded by business rule. Preserving that asymmetry is precisely what makes the dashboard total match Power BI; "fixing" it would break the reconciliation.

The failure this replaced

The first build differenced the register itself — `LAG()` over the cumulative column, a 15-minute gap cap, and a breaker-rating plausibility gate. It produced **1,780,590 kWh** against the warehouse's 2,020,697.5. The logic was defensible; it simply was not the company's. The gap was almost exactly the LONG_GAP mass the cap discarded.

---

03 · Classification

Where the thresholds come from

Every band the dashboard colours is read from the warehouse's own functions. None is a textbook value.

The `silver.classify_*_v2` family (19 functions) returns a text grade per reading. These are the actual definitions, taken from `pg_get_functiondef`:

FUNCTION	ACCEPTABLE	REVIEW REQUIRED	PROBLEMATIC	CRITICAL
`classify_power_factor_v2`	0.80 – 1.00	0.78 – 0.80	0.60 – 0.78	< 0.60 or > 1
`classify_voltage_neutral_v2`	215 – 245 V	209.94–215 245–250.06	207–209.94 250.06–253	< 207 or > 253
`classify_voltage_line_v2`	380 – 420 V	371.2–380 420–428.8	360–371.2 428.8–440	< 360 or > 440
`classify_frequency_v2`	59.3 – 60.5 Hz	—	59–59.3 60.5–61	< 59 or > 61

All four return `'Invalid'` for NULL. An early draft of the dashboard used 0.85 as the power-factor threshold — the Saudi tariff penalty point — which is a defensible industry value and the wrong one here. Corrected to 0.80.

A real gap in the status ladder

`silver.status_rank` ranks six labels. The three-argument `classify_current_v2` emits two more that it does not rank, so both fall through to its `ELSE 0` branch — *below* Acceptable:

LABEL	CONDITION	ROWS	RANKED?
`Major Issue`	Power recorded but current reads zero	34,069	NO

LABEL	CONDITION	ROWS	RANKED?
`Major Problem`	Current flowing but power reads zero	45,360	NO

These are sensor-disagreement faults — exactly the readings an engineer most wants to see. The dashboard originally hard-coded the six-label ladder and so dropped all **79,429** of them. The fix was to stop assuming the domain and discover it at startup:

```
SELECT DISTINCT t.v
FROM gold.energy_raw r
CROSS JOIN LATERAL (VALUES ('status_vn1', r.status_vn1), ...) AS t(col, v)
WHERE t.v IS NOT NULL;
```

Known values are kept in ladder order, anything new is appended and printed to the console at boot. If a classifier gains a label tomorrow, the status page shows it instead of silently discarding those rows.

RAISED FOR THE WAREHOUSE OWNER, NOT FIXED HERE

`status_rank` not ranking these two labels is harmless today because `worst_status` excludes the current channels. It would become a real defect the moment current is folded into the overall rollup, because a genuine fault would rank below `Acceptable`. That is a silver-layer change and was deliberately left alone by a read-only consumer.

---

04 · Server

Architecture and its constraints

One Python file, no framework, no ORM. Everything below is a deliberate constraint rather than an absence.

Read-only is enforced, then verified

```
c = psycopg.connect(DSN, row_factory=dict_row, connect_timeout=10)
c.execute('SET default_transaction_read_only = on')
c.commit()
mode = c.execute('SHOW transaction_read_only').fetchone()['transaction_read_only']
if mode != 'on':
    c.close()
    raise RuntimeError('refusing to serve: connection is not read-only')
```

Setting the flag is not enough — the setting is read back and the process refuses to serve if it did not take. The dashboard cannot write to the warehouse even if a query were malformed or hostile. Every query is parameterised (`%(name)s`); no user input is ever concatenated into SQL.

Threading model

`ThreadingHTTPServer` with one connection per thread, held in `threading.local()`. Each page issues 3–4 requests in a `Promise.all` ; under the original single-threaded server those serialised, so one slow aggregate left every panel on the page showing an em-dash. Each handler ends with `rollback()` to close the implicit transaction and avoid holding a snapshot open.

The dimension cache — the single biggest win

THE PROBLEM

`gold.meter`, `gold.zone` and `gold.facility` are views over silver staging views over bronze. Joining a 4.5M-row fact scan to them re-evaluates that entire stack per query. Measured: a **4-second** aggregate became **90 seconds**.

They are 31 and 21 rows. So they are resolved **once at startup** into a Python dict, facts are filtered on plain UUID keys, and zone/meter/facility labels are attached in Python afterwards. No fact query contains a join.

```
J_VAL = "FROM gold.energy_consumption_validated c"   -- no joins, deliberately
J_RAW = "FROM gold.energy_raw r"
```

The cost: a meter renamed in the warehouse is not reflected until the server restarts. For a static analytical snapshot that is an acceptable trade, and it is documented rather than hidden.

Response cache

Responses are memoised on the exact `path + query string`, capped at 400 entries and cleared wholesale on overflow. This is only safe because the warehouse is a static snapshot — there is no invalidation logic, and there deliberately is none to go wrong. **If the warehouse becomes live-loading, this cache must be removed or given a TTL.**

05 · Filters

Two builders, and why

The most subtle code in the system. Both correctness and performance live here.

There are two filter builders because the two fact sources carry time differently. `gold.energy_consumption_validated` has a real `reading_date_saudi` *date* column; `gold.energy_raw` has only `reading_timestamp_utc`.

`flt()` — the validated side

```
c.reading_date_saudi >= %(d1)s AND c.reading_date_saudi <= %(d2)s
c.energy_meter_key = ANY(%(mk)s::uuid[])          -- only when filtered
c.interval_quality  = ANY(%(iq)s)                 -- optional
```

`flt_raw()` — the raw side, where it gets interesting

```
r.reading_timestamp_utc >= (%(d1)s::date - interval '3 hours')
r.reading_timestamp_utc <  (%(d2)s::date + interval '1 day' - interval '3 hours')
```

WHY NOT THE OBVIOUS FORM

The natural way to write this is `(reading_timestamp_utc + interval '3 hours')::date BETWEEN d1 AND d2`. That is **not sargable** — wrapping the indexed column in an expression makes the planner discard `pk_energy_readings_snapshot` and scan the heap. Moving the arithmetic to the *constant* side leaves the bare column exposed to the index and returns exactly the same rows, because Saudi time is a fixed UTC+3 with no daylight saving. Measured: **103 ms → 28 ms**.

The counter-intuitive one: always name every meter

```
mk = keys_for(qs)
if mk is None:
    # Supply every key even when unfiltered.
    mk = [r['mk'] for r in DIM['meters']]
w.append('r.energy_meter_key = ANY(%(mk)s::uuid[])')
```

Adding a predicate that excludes nothing looks like waste. It is the opposite. The primary key is `(energy_meter_key, reading_timestamp_utc)` — meter first. A timestamp-only range cannot lead with the index and degrades to a sequential scan. Naming all 31 keys turns one 4.5M-row scan into 31 index range searches. Measured on a 7-day aggregate: **103 ms → 28 ms**.

A zone or meter filter returning an empty list short-circuits to the literal `FALSE` rather than issuing a query that cannot match.

06 · Time

The timezone model

One rule, applied everywhere, and the bug that proved it matters.

- **Storage is UTC.** `reading_timestamp_utc` is the only authoritative instant.

- **Business time is Saudi.** UTC+3, no DST, so the conversion is a fixed offset and never a lookup.
- **The warehouse's `reading_date_saudi` is authoritative** for day and month bucketing. Verified as exactly `(reading_timestamp_utc + interval '3 hours')::date` across all **4,260,849** rows, **zero** mismatches.
- **Two forms, one meaning.** Filtering shifts the *constant* (sargable, above); display shifts the *column* (`reading_timestamp_utc + interval '3 hours' AS ts`). Never the reverse.

WHY THIS EARNS A WHOLE SECTION

Power BI carried a calculated column `Reading Month UTC = DATE(YEAR([reading_timestamp_utc]), MONTH(...), 1)` — bucketing by UTC while every other part of the model, including its own weekday and hour logic, used Saudi. Monthly figures diverged from the dashboard by up to 1,251.9 kWh while *annual* totals matched exactly, because it was a boundary reallocation, not lost energy. Per-meter grand totals agreed to the decimal both ways, which is what identified it as a bucketing fault rather than a calculation one. Corrected in Power BI to Saudi.

07 · Endpoints

The fourteen routes

All under `/api/`, all GET, all JSON, all reading the two fact sources through the filter builders above.

ROUTE	SOURCE	RETURNS	NOTABLE TECHNIQUE
`/api/meta`	dims + bounds	Meter list, date bounds, thresholds	Served from the startup cache
`/api/kpi`	both	Headline cards	Two queries: consumption + electricals
`/api/daily`	validated	kWh per Saudi day	`FILTER` split by interval_quality
`/api/monthly`	validated	kWh per Saudi month	`to_char(reading_date_saudi,'YYYY-MM')`
`/api/zones`	validated	kWh per zone	Zone names joined in Python
`/api/meters`	both	Per-meter table	Two aggregates merged on meter key
`/api/series`	raw	Per-reading time series	Branches: single meter vs summed
`/api/readings`	raw	Individual readings	Paged; honours time-of-day filter
`/api/status`	raw	16 status columns × N labels	Single pass, `FILTER` grid
`/api/nonzero`	raw	Per-signal availability	Single pass, 4 counters × 26 signals
`/api/pfdist`	raw	PF histogram, 4 channels	`LATERAL VALUES` unpivot
`/api/quality`	gold views	Quality summary, gaps	Direct passthrough
`/api/bill`	gold.energy_bill	Invoice vs metered	Ordered by variance
`/api/sql`	—	The queries themselves	Transparency; re-runnable in psql

The technique that appears everywhere: one pass, many counters

The naive shape for "count each status label in each of 16 columns" is a `UNION ALL` of 16 grouped queries — sixteen reads of a 4.5M-row table. Conditional aggregation does it in one:

```
SELECT count(*) FILTER (WHERE status_vn1 = 'Acceptable') AS vn1_acceptable,
       count(*) FILTER (WHERE status_vn1 = 'Critical')   AS vn1_critical,
       ...                                  -- 16 columns x N labels
FROM gold.energy_raw r WHERE ...
```

Applied to `/api/status` (was 16 passes), `/api/nonzero` (26), and `/api/pfdist` (4). The PF histogram additionally unpivots four columns into four rows per reading so the heap is touched once:

```
SELECT t.col, width_bucket(t.v, -1, 1.2, 44) AS b, count(*)
FROM gold.energy_raw r
CROSS JOIN LATERAL (VALUES ('pf1', r.power_factor_phase_1),
                           ('pf2', r.power_factor_phase_2),
                           ('pf3', r.power_factor_phase_3),
                           ('pfa', r.power_factor_average)) AS t(col, v)
WHERE ... AND t.v IS NOT NULL
GROUP BY 1, 2;
```

44 buckets of 0.05 spanning −1 to 1.2. The range starts below zero on purpose: negative power factor is real in this data, and clipping the axis at zero was one of the things that made the first build misleading. Measured: 7.0 s → 1.6 s.

Cadence is measured, never assumed

Coverage needs an expected reading count. The nominal figure looks like 5 minutes; the true median inter-reading gap is **244 s (~4.07 min)**, about 354 readings per day. A hard-coded 288 understated the denominator by 23% and produced coverage figures above 100%. Each meter's own cadence is derived instead:

```
WITH g AS (
  SELECT energy_meter_key,
         EXTRACT(EPOCH FROM (reading_timestamp_utc
           - LAG(reading_timestamp_utc) OVER (PARTITION BY energy_meter_key
                                              ORDER BY reading_timestamp_utc))) AS gap_s
  FROM gold.energy_raw r WHERE ...)
SELECT energy_meter_key,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY gap_s) AS median_gap_s
FROM g WHERE gap_s IS NOT NULL AND gap_s > 0
GROUP BY 1;
```

Median rather than mean, so a single multi-day outage does not distort a meter's expected cadence.

Availability is two questions, not one

`/api/nonzero` reports `present` (NOT NULL) separately from `nonzero`. Every column except the kVA family is 100% NOT NULL, yet `reactive_energy_cumulative_kvarh` is exactly zero on well over half its rows. A single "availability" column conflated "always transmitted" with "always meaningful"; the five kVA columns are 0% transmitted, which is a different fact again from being zero.

08 · Front end

How the page is driven

One HTML file, no build step, no framework. Chart.js from CDN is the only dependency.

State and dispatch

A single `state` object holds the filters. Changing any control mutates it and calls `render()`, which dispatches through a map rather than a conditional chain:

```
var RENDER = {cm:renderCommand, en:renderEnergy, cu:renderCurrent, vf:renderVolt,
              pf:renderPF, rd:renderReadings, dq:renderDQ, bl:renderBill};
```

Each renderer issues its requests with `Promise.all` and draws only when all resolve, so a page never paints half-populated.

The staleness token — and the bug it was written to fix

```
var _renderSeq = 0;                  // incremented ONCE per render()
function api(path, extra){
  var mine = _renderSeq;
  return fetch(...).then(function(j){
      return {data:j, stale: mine !== _renderSeq};  // caller discards if stale
  });
}
```

THE SUBTLETY

The first implementation incremented the counter inside `api()`, i.e. once per *request*. Inside a `Promise.all` of four calls, each call bumped the counter, so every request but the last marked *itself* stale and the render bailed out with every panel showing an em-dash. The counter must advance once per render, not once per request — a one-line distinction with a total-failure symptom.

Chart conventions

- Labels carry the **full** timestamp; the axis abbreviates via a tick callback. Truncating at source would make tooltips ambiguous.
- Phase colours are fixed across every page: L1 / L2 / L3 always map to the same three CSS variables, so a reader never re-learns the legend.
- Zoom and pan are enabled on the x-axis only, with double-click to reset.
- Current unbalance uses NEMA MG-1 / IEEE 141 — `max|phase â^’ mean| / mean` — blanked below 0.5 A mean, where the ratio is noise. The earlier `(max â^’ min) / max` was a different quantity: capped at 100%, blind to which phase deviates, and inflated on lightly-loaded boards.

A Chart.js trap worth knowing

One chart is horizontal (`indexAxis:'y'`). On such a chart `parsed.y` is the **category index** and `parsed.x` is the value — the reverse of every other chart. A shared tooltip that preferred `parsed.y` therefore printed the row number, identically for every series. The same inversion hit the axis: the shared `ticks.callback` formats numbers, and on a category axis it is handed the index, so signal names rendered as `0, 3.0, 6.0`. Both now branch on `indexAxis`.

09 · Validation

The method, reproducibly

Three independent levels. A figure was accepted only when all three agreed.

Level 1 — against Power BI

All 52 DAX measures re-computed in SQL. Every one reconciles. Two observations recorded as findings rather than defects: `meter_label` on `gold energy_raw` is a calculated column (`RELATED('gold meter'[meter_label])`) rather than a view column, and the model omits the `apparent_power_*` , `apparent_energy_cumulative_kvah` and both `*_average_v` columns — which the availability analysis later confirmed are 0% transmitted, so the omission was correct.

Level 2 — against bronze, bypassing the warehouse

The strongest check. Rather than trust the pipeline, the raw signals were re-pivoted independently and the dashboard's figures recomputed. Channel ids are resolved at runtime, never hard-coded:

```
SELECT id, key FROM bronze.telemetries_dictionaries
WHERE key IN ('Amp1','Amp2','Amp3');        -- e.g. 17, 33, 34

WITH bounds AS (
  SELECT (extract(epoch from (%(d1)s::date)::timestamp
          AT TIME ZONE 'Asia/Riyadh')*1000)::bigint AS lo, ...)
SELECT t."timestamp",
       max(round(t.dbl::numeric,1)) FILTER (WHERE t.key_id = 17) AS a1, ...
FROM bronze.telemetries t CROSS JOIN bounds
WHERE t.device_id = %(mk)s::uuid AND t.key_id = ANY(%(kids)s)
  AND t."timestamp" >= bounds.lo AND t."timestamp" < bounds.hi
GROUP BY t."timestamp";
```

Then `FULL JOIN` ed to `gold.energy_raw` on the timestamp. Result for the sample meter and window:

```
readings compared        1,579      channels per reading   3
in bronze not in gold        0      amp rows           4,737  = 1,579 x 3
in gold not in bronze        0      rows changed by rounding   0
values that disagree         0
```

Repeated for current, voltage, frequency, power, power factor and consumption. Comparing totals alone would not have caught offsetting errors; the row-level FULL JOIN would.

Level 3 — physics

Two checks that an internally-consistent error cannot satisfy:

```
line-to-line / phase-to-neutral   1.731836     vs âˆš3 = 1.732051  (0.012% error)
register delta vs consumption     710.8 kWh    vs validated 710.8 kWh  (exact)
```

The second is the strongest single check in the system: the meter's cumulative counter is independent of every calculation in the warehouse, so agreement is not something a pipeline bug could manufacture.

WHY "THE TOTALS MATCH" IS WEAK ON ITS OWN

Two reports summing the same column will always agree, whatever that column contains. Only levels 2 and 3 test the column itself. When a discrepancy did appear — the UTC month bug — it was the *pattern* (annual totals identical, per-meter totals identical, monthly figures offsetting between adjacent months) that identified it as bucketing rather than arithmetic.

---

10 · Debt

Known limitations and traps

Stated plainly, so nobody rediscovers them the hard way.

ITEM	CONSEQUENCE	IF YOU CHANGE IT
Dimension cache loaded once at startup	Meter or zone renames need a restart	Add a reload endpoint, or a TTL
Response cache has no invalidation	Safe only while the warehouse is a static snapshot	**Must** be removed or given a TTL if loading becomes live
Time-of-day filter reaches only `/api/readings`	KPI cards ignore the chip shown in the filter bar	Thread `t1` / `t2` through `flt_raw()`
Single-process, localhost only	Not reachable by colleagues	Front with a real WSGI server before exposing it
DSN password has an in-code default	Convenience for a local tool	Move to `PGPASSWORD` / a secret store before any network exposure
`/api/readings` capped at 20,000 rows	Very large windows truncate	Cap is reported in the response, not silent
`status_rank` omits two labels	Latent, harmless today	Silver-layer fix; see section 03
kVAr channel unreliable at ZN.15-PB.2	Source data, not pipeline	Check other meters before trusting reactive power

11 · Extending

Adding to the system

A new endpoint

- Write `def ep_thing(qs)`; call `flt(qs)` or `flt_raw(qs)` for the WHERE fragment and params — never build a predicate by hand.
- Return through `clean()` so Decimal, date and timedelta serialise correctly.
- Register it in `ROUTES`. Caching, error handling and JSON encoding are inherited.
- Add zone/meter context in Python from `DIM`; do not join to `gold.meter`.

A new page

- Add an entry to `PAGES`, markup in `buildPages()`, a `renderX()` function, and one line in `RENDER`.
- Use `api()` and honour the `stale` flag, or stale responses will overwrite fresh ones.
- Take colours from CSS variables so both themes and the print stylesheet keep working.

Verifying a change

Re-run the level-2 bronze comparison for any figure you touch. Totals agreeing is not sufficient evidence — that was the lesson of the UTC month bug, where every total agreed and the monthly figures were still wrong.

SADIN Energy Intelligence — engineering reference, 15 September 2026. Companion to the project record. Line counts, row counts and thresholds were read from the codebase and the production catalogue on the date of writing.